

Práctica 1 – Optimización Estocástica CM0856

Ruteo urbano con tiempos de viaje estocásticos en Manhattan

Salomón Mejía Isaza

Salomón Cardeño Luján

Maestría en Matemáticas Aplicadas — Universidad EAFIT

15/09/2026

Resumen

Se modela y resuelve un programa estocástico lineal de dos etapas para el ruteo de una cuadrilla municipal en Manhattan, con tiempos de viaje inciertos estimados a partir de los registros oficiales de taxis amarillos de la ciudad de Nueva York (TLC). La primera etapa decide la ruta mediante variables binarias de arco con eliminación de subtours de Miller–Tucker–Zemlin; la segunda etapa, continua, reparte la carga de trabajo entre ejecución directa, tercerización, tiempo adicional ordinario y sobretiempo de emergencia. Se construye una aproximación por promedio muestral (SAA), se valida una formulación extensa como programa lineal entero mixto (MILP) contra una implementación propia del algoritmo *Integer L-shaped*, y se contrasta la solución estocástica con la aproximación determinista del perfil promedio de tiempos, tanto dentro de muestra como en una validación temporal fuera de muestra con datos de febrero de 2015. La construcción de la muestra ya se validó contra el documento oficial: 110 pools origen–destino y 242 843 registros filtrados sobre 12 741 035 registros crudos de enero de 2015. La implementación quedó validada de forma exacta contra la forma extensa MILP y contra la forma cerrada del recurso. En el experimento principal con $K = 200$ escenarios, la ruta óptima estocástica x^* resultó ser idéntica a la aproximación determinista x_{prom} del perfil promedio, de modo que $VSS_K = 0$ dentro de muestra; la validación temporal con 1000 escenarios de febrero de 2015 confirma esa misma conclusión ($\Delta_{\text{Feb}} = 0$, con un intervalo pareado del 95 % que colapsa al punto $(0, 0)$), evidencia de que la decisión de primera etapa es robusta frente a la forma de modelar la incertidumbre en esta instancia. El valor objetivo certificado del experimento principal es $UB = 50.555952$ (cota inferior $LB = 50.515598$, brecha relativa $7.982 \times 10^{-4} \leq 10^{-3}$), obtenido en 599.42 s tras explorar 1673 nodos y generar 1975 cortes de Benders más 1 corte entero.

1. Descripción del problema y modelo matemático

1.1. Descripción y contexto

Una cuadrilla municipal, con un único vehículo, debe partir de un depósito, visitar todos los sitios indicados y regresar al depósito. El orden de visita se decide antes de conocer el tráfico del día. Una vez se revelan los tiempos de viaje, la cuadrilla decide cuántos minutos de trabajo realiza directamente en cada sitio, cuántos minutos terceriza y cuánto tiempo adicional utiliza.

El horizonte H representa el tiempo efectivo disponible para desplazamientos y trabajo de campo. No corresponde a la duración completa del turno: se supone que pausas, alistamiento y labores administrativas ya fueron descontadas. Existe un primer tramo limitado de tiempo adicional y, para garantizar recurso completo, un sobretiempo de emergencia más costoso.

1.2. Modelo matemático

En primer lugar, cabe aclarar que el documento oficial con el enunciado de la práctica explícitamente indica que el problema se trata de un problema estocástico lineal de dos etapas. La primera etapa contiene decisiones binarias de ruteo (decidir el orden de visita antes de conocer el tráfico del día), la incertidumbre se encuentra en los tiempos de viaje, y la segunda etapa contiene únicamente decisiones continuas de recurso (decidir cuántos minutos de trabajo realiza directamente la cuadrilla, cuántos terceriza, cuánto tiempo adicional, etc.).

Por otro lado, el documento también propone una notación particular para el problema, así como la definición de variables, parámetros y restricciones. De acuerdo con esto, podemos formular el problema a continuación.

1.2.1. Índices, parámetros, variables y restricciones

Conjuntos e índices.

$$V = \{0, 1, \dots, 10\}, \quad N = V - \{0\}, \quad A = \{(i, j) \in V \times V : i \neq j\}, \quad |A| = 110, \quad n = |N| = 10.$$

Donde V es el conjunto de todos los índices de cada uno de los 10 sitios incluyendo el del depósito (0), N es el conjunto de índices sin incluir el del depósito, A es el conjunto de arcos que simboliza la ruta entre dos sitios, $|A| = 110$ es el número de arcos resultante por los datos y $n = |N| = 10$ es simplemente el número de sitios distintos del depósito.

Parámetros.

- $c_{ij} = \kappa d_{ij}^{\text{road}}$: costo de desplazamiento del arco $(i, j) \in A$ en [USD], donde $\kappa = 1$ USD/km es el costo de desplazamiento y d_{ij}^{road} es la distancia determinista por arco en kilómetros.
- d_i : demanda de minutos de servicio a cumplir en el sitio $i \in N$ [min].
- c_i^{out} : costo de tercerizar el servicio en el sitio $i \in N$ [USD/min].
- $H = 390$ min: horizonte regular de tiempo [min].
- $\bar{o} = 30$ min: tiempo adicional ordinario disponible [min].
- $c^{\text{OT}} = 1.5$ USD/min: costo unitario de *overtime* (tiempo adicional ordinario) [USD/min].
- $c^{\text{EM}} = 6$ USD/min: costo unitario de *emergency* (sobretiempo de emergencia) [USD/min].
- $\alpha_i = 1$: productividad en el sitio $i \in N$.
- $\xi_{ij}^{(s)}$: tiempo de viaje aleatorio del arco $(i, j) \in A$ en el escenario s [min].

Variables de primera etapa.

- $x_{ij} \in \{0, 1\}$: la variable binaria que indica si la ruta usa el arco $(i, j) \in A$ o no.
- $v_i \in [1, n]$: variable auxiliar que indica el orden en el que se visita el sitio $i \in N$ (**Nota:** se puede asumir continua sin afectar la modelación, pues asumirla entera no aporta en nada).

Restricciones de ruteo (primera etapa). La ruta debe salir una vez de cada nodo y entrar una vez a cada nodo. Como una forma natural de ilustrar o imaginar las rutas es a través de grafos dirigidos (con los sitios como nodos y las aristas como las rutas), esta restricción suele llamarse **restricción de grado** en problemas de ruteo, pues básicamente cuenta el número de aristas conectadas a un nodo y, en caso de ser dirigidas, cuántas entran y cuántas salen (como en nuestro caso). Por lo tanto, en nuestro caso tenemos una única arista entrante y saliente para cada nodo, lo cual se puede modelar como

$$\sum_i x_{ij} = 1, \quad \forall j \in V \quad (\text{entrar una vez a cada nodo}), \quad (1)$$

$$\sum_j x_{ij} = 1, \quad \forall i \in V \quad (\text{salir una vez de cada nodo}). \quad (2)$$

La ruta debe excluir subrutas desconectadas (*subtours*) mediante las restricciones de Miller–Tucker–Zemlin (MTZ). De acuerdo con los autores [3], nuestro problema de ruteo en la primera etapa se puede ver como una Programación Entera del *Travelling Salesman Problem* (TSP). En particular, esto implica que debemos considerar restricciones de grado (que ya consideramos anteriormente) y restricciones para eliminar subtours de rutas con ciclos (o *loops*) que cumplen las restricciones de grado, pero que no nos garantizan que la única ruta con ciclo sea la que necesitamos: partir del depósito, pasar por todos los demás sitios una vez y regresar al depósito. Siguiendo nuestra notación, esto lo podemos modelar como

$$v_i - v_j + n x_{ij} \leq n - 1, \quad \forall i, j \in N, \quad i \neq j \quad (\text{eliminar loops excepto el del depósito}). \quad (3)$$

La lógica de los autores es la siguiente:

- si un arco no se usa en la ruta $x_{ij} = 0$ la restricción se vuelve $v_i - v_j \leq n - 1$. Como $v_i, v_j \in [1, n]$ el máximo valor posible de esa diferencia es $n - 1$, entonces en este caso la restricción se cumple automáticamente, i.e., no impone nada sobre el problema.

- si un arco se usa en la ruta $x_{ij} = 1$ la restricción se vuelve $v_i - v_j + n \leq n - 1 \implies v_j \geq v_i + 1$. Esto quiere decir que si viajamos directamente de i a j , el orden en el que visitamos a j debe ser como mínimo uno más que el orden en el que visitamos a i . Precisamente, esta propiedad de crecimiento estricto es la que muestra que ciclos cerrados de *subtours* sea imposible pues, hacer ciclos cerrados en *subtours* que no incluyen al depósito forzarían a que v incremente a lo largo del tour y al mismo tiempo, de alguna manera, logre volver al nodo inicial, lo cual es una contradicción. Por esta razón, este caso es el que elimina *loops* de *subtours* que no sean el del depósito.

La ruta debe partir del depósito, visitar todos los sitios y regresar al depósito. Note que, precisamente, esto ya lo garantizan las dos restricciones anteriores: las de grado incluyen el nodo 0 como entrada/salida y las MTZ garantizan por eliminación que la única ruta con ciclo sea la que queremos (depósito como partida y fin).

Variables de segunda etapa. Para cada escenario $s = 1, 2, \dots, K$ sean

- $u_i^{(s)} \in [0, d_i]$: cantidad de minutos de trabajo directo (por la cuadrilla) en el sitio $i \in N$ [min].
- $r_i^{(s)} \geq 0$: cantidad de minutos de trabajo tercerizados en el sitio $i \in N$ [min].
- $o^{(s)} \in [0, \bar{o}]$: tiempo adicional ordinario [min].
- $e^{(s)} \geq 0$: sobretiempo de emergencia [min].

Note que $u_i^{(s)}$ y $o^{(s)}$ son acotados, por lo que $r_i^{(s)}$ y $e^{(s)}$ no se acotan superiormente para garantizar recurso completo.

Restricciones de recurso (segunda etapa). Al fijar una ruta x y ocurrir el escenario $\xi_s = \xi_{ij}^{(s)}$, debemos encontrar el valor óptimo del subproblema $Q(x, \xi_s)$ sujeto a

La cantidad de minutos de trabajo directo y tercerizado en cada sitio, abastece el tiempo de trabajo demandado en cada sitio. Esto se modela como

$$u_i^{(s)} + r_i^{(s)} = d_i, \quad \forall i \in N \quad (\text{tiempo ejecutado cubre el demandado}). \quad (4)$$

El tiempo de viaje de la ruta y el de trabajo directo no puede superar el tiempo máximo disponible. Esto se modela como

$$\sum_{(i,j) \in A} \xi_{ij}^{(s)} x_{ij} + \sum_{i \in N} \alpha_i u_i^{(s)} \leq H + o^{(s)} + e^{(s)} \quad (\text{tiempo usado no supera el disponible}). \quad (5)$$

Función objetivo del subproblema de segunda etapa. El objetivo del subproblema de la segunda etapa es minimizar los costos obtenidos por tercerización, uso de tiempo adicional ordinario y uso de sobretiempo de emergencia. Entonces la función objetivo de cualquier subproblema debe ser

$$\sum_{i \in N} c_i^{\text{out}} r_i^{(s)} + c^{\text{OT}} o^{(s)} + c^{\text{EM}} e^{(s)}. \quad (6)$$

1.3. Formulación del SIP

Siguiendo todo lo anterior, podemos formular el modelo matemático del *Stochastic Integer Problem* (SIP) con primera etapa entera como

$$\begin{aligned} \min_{x,v} \quad & \sum_{(i,j) \in A} c_{ij} x_{ij} + Q(x) \\ \text{s.a.} \quad & (1), (2), (3), \\ & x_{ij} \in \{0, 1\}, \quad v_i \in [1, n]. \end{aligned} \quad (7)$$

Donde $Q(x) = \mathbb{E}[Q(x, \xi_s)]$ y

$$\begin{aligned} Q(x, \xi_s) = \min_{u,r,o,e} \quad & \sum_{i \in N} c_i^{\text{out}} r_i^{(s)} + c^{\text{OT}} o^{(s)} + c^{\text{EM}} e^{(s)} \\ \text{s.a.} \quad & (4), (5), \\ & u_i^{(s)} \in [0, d_i], \quad r_i^{(s)} \geq 0, \quad o^{(s)} \in [0, \bar{o}], \quad e^{(s)} \geq 0. \end{aligned}$$

1.4. Formulación del problema SAA

Usando *Sample Average Approximation* (SAA) podemos aproximar muestralmente a la función de recurso $Q(x) = \mathbb{E}[Q(x, \xi_s)]$ como

$$Q_K(x) := \frac{1}{K} \sum_{s=1}^K Q(x, \xi_s). \quad (8)$$

Entonces, reemplazando $Q_K(x)$ en la formulación anterior obtenemos

$$\begin{aligned} \min_{x,v} \quad & \sum_{(i,j) \in A} c_{ij} x_{ij} + Q_K(x) \\ \text{s.a.} \quad & (1), (2), (3), \quad x_{ij} \in \{0, 1\}, \quad v_i \in [1, n]. \end{aligned} \quad (9)$$

Donde $Q(x, \xi_s)$ es el subproblema definido en (5)–(6).

1.5. Formulación del MILP

Siguiendo la formulación del problema SAA anterior, podemos escribir todo en una única formulación extensa, i.e., la formulación del *Mixed-Integer Linear Programming* (MILP). Esto es

$$\begin{aligned} \min_{x,v,u^{(s)},r^{(s)},o^{(s)},e^{(s)}} \quad & \sum_{(i,j) \in A} c_{ij} x_{ij} + \frac{1}{K} \sum_{s=1}^K \left[\sum_{i \in N} c_i^{\text{out}} r_i^{(s)} + c^{\text{OT}} o^{(s)} + c^{\text{EM}} e^{(s)} \right] \\ \text{s.a.} \quad & (1), (2), (3), \\ & u_i^{(s)} + r_i^{(s)} = d_i, \quad \forall i \in N, \quad \forall s = 1, \dots, K, \\ & \sum_{(i,j) \in A} \xi_{ij}^{(s)} x_{ij} + \sum_{i \in N} \alpha_i u_i^{(s)} \leq H + o^{(s)} + e^{(s)}, \quad \forall s = 1, \dots, K, \\ & x_{ij} \in \{0, 1\}, \quad v_i \in [1, n], \quad u_i^{(s)} \in [0, d_i], \quad r_i^{(s)} \geq 0, \quad o^{(s)} \in [0, \bar{o}], \quad e^{(s)} \geq 0. \end{aligned} \quad (10)$$

1.6. Tamaño del modelo

1.6.1. SIP y SAA

Note que ambas formulaciones están escritas como problemas anidados por etapa: problema de primera etapa sobre x y v y subproblema de segunda etapa sobre u , r , o y e , que se resuelve cuando se decide la ruta (x fijo). Por esta razón, resulta más útil pensar en el total de variables del problema de primera etapa y el de cada subproblema por escenario en la segunda etapa:

- **Primera etapa:** 110 binarias (x_{ij}) + 10 continuas (v_i) = 120 variables.
- **Subproblema, por cada escenario:** 10 continuas ($u_i^{(s)}$) + 10 continuas ($r_i^{(s)}$) + 1 continua ($o^{(s)}$) + 1 continua ($e^{(s)}$) = 22 variables por escenario $s = 1, \dots, K$.

1.6.2. MILP

Siguiendo la lógica anterior, observe que en esta formulación extensa hay básicamente K copias del subproblema para cada escenario proveniente de la SAA. Entonces el número de variables son 110 binarias + (10 + 22 K) continuas. Note que:

- Para la prueba de consistencia con $K = 50$, esto resulta en $110 + 10 + 22(50) = 1220$ variables.
- Para el experimento principal con $K = 200$, esto resulta en $110 + 10 + 22(200) = 4410$ variables.
- Ninguna de las variables de la primera etapa aumenta, en particular, las binarias se mantienen fijas. Únicamente las variables continuas aumentan a medida que crece el número de escenarios K .

1.7. Una propiedad estructural del recurso

Además de la formulación anterior, conviene señalar una propiedad del subproblema de segunda etapa que se usa más adelante para implementar y validar el Integer L-shaped (sección 4). El escenario entra en el subproblema (6)–(5) *únicamente* a través del escalar

$$\tau_s(x) = \sum_{(i,j) \in A} \xi_{ij}^{(s)} x_{ij}, \quad (11)$$

que es el tiempo total de viaje de la ruta en ese escenario. Todos los demás datos del subproblema — d_i , c_i^{out} , H , $\bar{o}$, c^{OT} , c^{EM} — son comunes a todos los escenarios. Por lo tanto existe una *única* función $\varphi : \mathbb{R} \rightarrow \mathbb{R}_{\geq 0}$, convexa, no decreciente y lineal por tramos, tal que

$$Q(x, \xi_s) = \varphi(\tau_s(x)) \quad \text{para todo } s. \quad (12)$$

La forma de φ se obtiene por un argumento de costo marginal. Si $\tau \leq H - \sum_i d_i = 390 - 310 = 80$, toda la demanda cabe en el horizonte regular y $\varphi(\tau) = 0$. En otro caso hay un déficit $\delta = \tau + 310 - H$ minutos que debe cubrirse con la combinación más barata de las tres válvulas de escape, que por ser un programa lineal con una única restricción de acoplamiento se llena de forma voraz en orden creciente de costo marginal: primero $\bar{o} = 30$ minutos de tiempo adicional a 1.50 USD/min, luego la tercerización sitio por sitio en orden creciente de c_i^{out} hasta agotar los 310 minutos de demanda, y finalmente la emergencia a 6.00 USD/min sin límite. Así, $\varphi'(\tau) = -\lambda_s$ es exactamente el costo marginal del tramo en el que cae el déficit, donde $\lambda_s \leq 0$ es el multiplicador de la restricción (5).

Esta propiedad se explota de dos maneras. Computacionalmente, permite construir los K modelos de segunda etapa una sola vez y reoptimizar cambiando únicamente el lado derecho de (5), en lugar de reconstruirlos en cada separación de cortes. Metodológicamente, proporciona una verificación independiente de la implementación: los valores $Q(x, \xi_s)$ y los multiplicadores λ_s entregados por el *solver* deben coincidir con $\varphi(\tau_s)$ y con el costo marginal teórico (sección 4.4).

2. Entorno computacional y código

2.1. Entorno computacional

La implementación computacional de la práctica se realizó en una instancia gratis de Google Colab en un notebook de Python, que básicamente crea un ambiente virtual en un *cloud server* que usa el *Python 3 Google Compute Engine backend* y aloca temporalmente los siguientes recursos computacionales para el usuario. Hay distintos tipos de *runtime* dependiendo de la necesidad de cada usuario y en nuestro caso usamos el que está por defecto hoy (2026-2): **Backend:** *Python 3 Google Compute Engine backend*; **CPU:** x2 AMD EPYC 7B12 @2250 MHz; **RAM:** 12.7 GB; **Disk:** 107.7 GB; **Python:** 3.13.15.

2.2. Código

El código utilizado se organiza en tres módulos —`p1_datos.py`, `p1_modelo.py` y `p1_experimentos.py`— embebidos en un único notebook autocontenido, disponible en un repositorio de GitHub con instrucciones detalladas para ejecutarlo y reproducir los resultados presentados:

https://github.com/scardenol/Stochastic_Optimization_2026/tree/main/Pr%C3%A1ctica%201

A modo de referencia, el fragmento de código en el apéndice B.1 muestra la construcción de los pools origen–destino (módulo `p1_datos.py`), que se describe con detalle en la sección 3.

3. Datos, filtros, pools, control de calidad y escenarios

3.1. Fuente de los datos

Los datos son el archivo oficial *Yellow Taxi Trip Records* correspondiente a los registros de enero de 2015, publicado por la *Taxi & Limousine Commission* (TLC) de la ciudad de Nueva York, que es la entidad municipal que regula los servicios de taxi y vehículos de alquiler de la ciudad. La entidad publica los registros de viaje [1] y, en particular, los de enero de 2015 se encuentran disponibles en:

El archivo resultante es un archivo en formato **parquet** y tiene un tamaño aproximado de 175 MB.

3.2. Nodos y zonas TLC

La TLC divide la ciudad en zonas geográficas numeradas para registrar los orígenes y destinos de los viajes. Cada coordenada de un sitio se representa mediante la zona que contiene, cuya etiqueta numérica aparece en la columna **LocationID**. En esta práctica consideramos únicamente 1 nodo depósito y 10 nodos sitios, todos pertenecientes a zonas distintas. La tabla 1 contiene toda la información sobre los sitios.

Tabla 1: Sitios de la instancia y su zona TLC.

i	Sitio	Latitud	Longitud	LocationID
0	Depósito: Javits Center	40.75750	-74.00250	246
1	Times Square	40.75800	-73.98550	230
2	Rockefeller Center	40.75870	-73.97870	161
3	Grand Central Terminal	40.75278	-73.97722	162
4	New York Public Library (Main)	40.75306	-73.98194	164
5	Union Square	40.73590	-73.99110	234
6	Washington Square Park	40.73083	-73.99750	114
7	Madison Square Garden	40.75056	-73.99361	186
8	One World Trade Center	40.71274	-74.01338	261
9	New York Stock Exchange	40.70693	-74.01125	87
10	South Street Seaport (Pier 17)	40.70600	-74.00270	209

3.3. Carga de datos y selección de columnas

Para cargar los datos, se utilizó la librería **polars** para leer el archivo en formato **parquet** una vez que fue descargado directamente en la sesión virtual de Colab. Esto resulta en un data frame con dimensiones 12741035 filas $\times$ 19 columnas, y un tamaño estimado de 1567.47 MB o 1.57 GB en memoria.

De las 19 columnas disponibles se seleccionaron únicamente las columnas relevantes:

tpep_pickup_datetime, **tpep_dropoff_datetime**, **PULocationID**, **DOLocationID** y **trip_distance**.

3.4. Filtrado de datos

De los 12741035 registros, únicamente se conservan los registros que satisfacen simultáneamente:

1. la fecha de recogida ocurrió entre el 1 y el 31 de enero de 2015, de lunes a viernes y en la franja horaria [09:00, 17:00);
2. el origen y el destino pertenecen a dos zonas distintas de la tabla;
3. la duración $t = (\text{tpep_dropoff_datetime} - \text{tpep_pickup_datetime})/60$ está entre 1 y 90 minutos;
4. la distancia del viaje **trip_distance** está entre 0.1 y 30 millas.

Dos decisiones de implementación merecen mención por su efecto en la trazabilidad. Primero, el límite superior de la ventana de fechas se impone como *exclusivo* en 2015-02-01 y no como un límite inclusivo en 2015-01-31, que perdería el último día salvo el instante de medianoche; en enero el efecto es nulo porque el 31 fue sábado y el filtro de días hábiles ya lo elimina, pero la forma exclusiva es la correcta y se reutiliza sin cambios para la validación de febrero. Segundo, la duración se calcula en minutos con parte fraccionaria (**total_seconds()/60**) y no truncando a minutos enteros.

Después de aplicar los filtros, pasamos de 12741035 registros crudos a **242 843 registros resultantes**.

3.5. Construcción y control de calidad de los pools

- **Conjunto de datos filtrados:** para cada arco $(i, j) \in A$, sea R_{ij} el conjunto de registros que superan los filtros, tienen como origen la zona i y como destino la zona j .

- **Conjunto de registros históricos de tiempos por arco:** a partir de los registros filtrados resultantes, se define un *pool* como el conjunto de registros históricos

$$P_{ij} = \{t_r : r \in R_{ij}\},$$

donde t_r es la duración en minutos del viaje histórico $r \in R_{ij}$.

- **Conjunto de *pools*:** está definido como $\{P_{ij} : (i, j) \in A\}$.

La implementación computacional de los pools se hace agrupando los datos por PULocationID y DOLocationID, agregando por tiempo de duración t (columna P_ij), por longitud o conteo (columna textttregistros), y por la distancia determinista por arco (en kilómetros), definida como

$$d_{ij}^{\text{road}} = 1.60934 \times \text{mediana}\{\text{trip_distance}_r : r \in R_{ij}\} [\text{km}], \quad c_{ij} = \kappa d_{ij}^{\text{road}}.$$

En el apéndice B.2 se muestra la implementación de la función `control_calidad()`. Al realizar el control de calidad se obtienen $|A| = 110$ pools y el tamaño o número de registros del pool más pequeño fue de $|P_{ij}| = 87$ duraciones, por lo que se logran validar estos resultados con el documento oficial de la práctica (que exige al menos 50 registros por pool, con un mínimo esperado de 87).

Tabla 2: Control de calidad de la construcción de pools (enero de 2015).

Indicador	Esperado	Obtenido
Número de pools $ A $	110	110
Registros del pool más pequeño	87	87
Registros totales tras los filtros	—	242 843

Los tres controles de calidad coinciden exactamente con lo esperado por el documento oficial de la práctica.

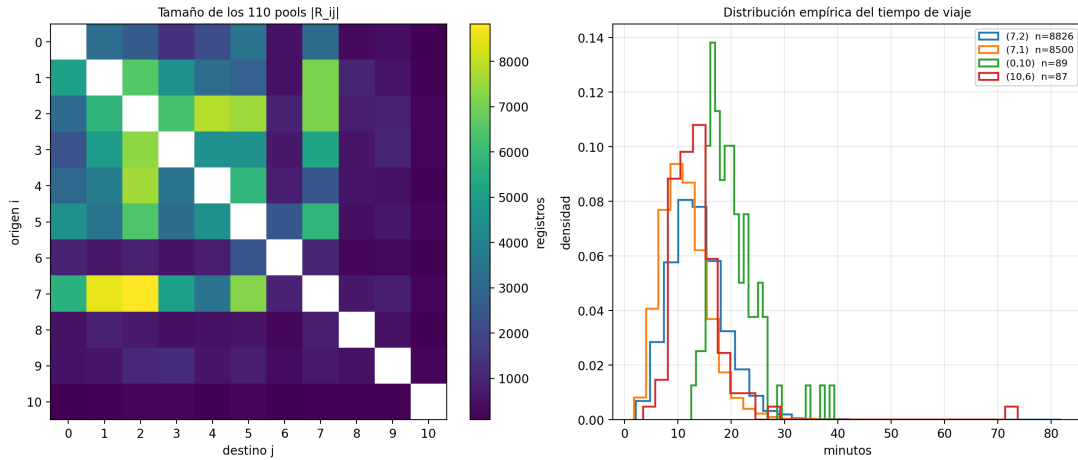


Figura 1: Izquierda: matriz de tamaños $|R_{ij}|$ de los 110 pools. Derecha: distribución empírica del tiempo de viaje para cuatro arcos representativos.

La matriz de tamaños es marcadamente heterogénea. Los *pools* más grandes (en torno a 7 000–9 000 registros) son los que conectan el clúster de Midtown —Times Square (1), Rockefeller Center (2), Madison Square Garden (7), Grand Central (3) y la biblioteca pública (4)— entre sí: son zonas contiguas con altísimo volumen de recorridos cortos. Los *pools* más pequeños (decenas de registros, visibles como celdas casi negras) involucran al depósito (0) y a los sitios del bajo Manhattan —Washington Square Park (6), One World Trade Center (8), la Bolsa de Nueva York (9) y South Street Seaport (10)—, que están geográficamente aislados del resto y reciben muchos menos viajes registrados por la TLC en la franja horaria filtrada.

El panel derecho ilustra esa misma asimetría en las distribuciones. Los arcos de alto volumen, (7, 2) con $n = 8 826$ y (7, 1) con $n = 8 500$, tienen distribuciones unimodales y relativamente concentradas, con moda entre 8 y 12 minutos y colas cortas. Los arcos de bajo volumen, (0, 10) con $n = 89$ y (10, 6) con $n = 87$, son visiblemente más ruidosos: (0, 10) tiene una distribución más ancha y desplazada hacia tiempos mayores (moda cerca de 15 minutos, con densidad apreciable hasta 35 minutos), y (10, 6) muestra una cola larga hacia

la derecha con un valor atípico aislado cerca de 75 minutos. Esto es consistente con lo esperado: con pocas observaciones, un solo viaje inusual (tráfico, una ruta no directa) pesa mucho más en la forma empírica del *pool*, y por eso estos arcos de cola pesada son también los que más presionan el lado derecho del recurso de segunda etapa cuando aparecen en la ruta óptima.

3.6. Generación de escenarios

En general, podemos representar la incertidumbre en nuestro problema como el vector aleatorio

$$\xi = (\xi_{ij})_{(i,j) \in A} \in \mathbb{R}^{110},$$

donde ξ_{ij} es el tiempo de viaje aleatorio, en minutos, del arco dirigido (i, j) . Cada escenario s es básicamente una realización de dicho vector $\xi_s = (\xi_{ij}^{(s)})_{(i,j) \in A}$.

Para generar K escenarios para cada arco (i, j) , se hace un muestreo aleatorio con reemplazo de cada *pool* P_{ij} correspondiente. Esto equivale a pensar $\xi_{ij}^{(s)} \sim \hat{F}_{ij} =$ distribución empírica de P_{ij} , de forma independiente entre arcos. En consecuencia ξ_s representa una realización *sintética* del tráfico y no la reconstrucción literal de un día calendario (la sección 5.4 discute lo que esta hipótesis descarta). Computacionalmente, esto se implementa con una función `muestrear_escenarios()` presentada en el apéndice B.3.

4. Implementación y validación del Integer L-Shaped

El problema SAA no puede abordarse directamente con el método L-shaped clásico porque las variables de primera etapa son binarias: la función de recurso esperada solo se aproxima correctamente por cortes en los puntos enteros. Se implementa entonces el *Integer L-shaped* de Laporte y Louveaux [4], que combina un esquema de *branch-and-bound* sobre x con la descomposición L-shaped dentro de cada nodo.

4.1. Descripción del algoritmo

El maestro. En cada nodo ν del árbol se resuelve el programa *lineal*

$$\begin{aligned} \min_{x, v, \theta} \quad & \sum_{(i,j) \in A} c_{ij} x_{ij} + \theta \\ \text{s.a.} \quad & (1)-(3) \quad (\text{ruteo relajado, } 0 \leq x_{ij} \leq 1), \\ & \ell_{ij}^\nu \leq x_{ij} \leq u_{ij}^\nu \quad (\text{cotas locales de ramificación}), \\ & \theta \geq L = 0, \\ & \text{todos los cortes globales acumulados,} \end{aligned}$$

cuya solución se denota (x^ν, θ^ν) . La variable θ aproxima $Q_K(x)$ desde abajo.

Los subproblemas y los cortes de Benders. Dado x^ν , posiblemente fraccional, se evalúan los K subproblemas (6)–(5) y se recuperan los multiplicadores λ_s asociados a la restricción de tiempo (5). Por la propiedad (12), la desigualdad de subgradiente para la función convexa φ da

$$Q(x, \xi_s) \geq Q(x^\nu, \xi_s) - \lambda_s (\tau_s(x) - \tau_s(x^\nu)).$$

Promediando sobre los K escenarios con probabilidad $1/K$ se obtiene el **corte estándar de Benders**

$$\theta \geq \underbrace{\frac{1}{K} \sum_{s=1}^K \left(Q(x^\nu, \xi_s) + \lambda_s \tau_s(x^\nu) \right)}_{\text{constante}} - \underbrace{\frac{1}{K} \sum_{s=1}^K \lambda_s \xi^{(s)} \cdot x}_{\text{coeficientes}}. \quad (13)$$

Como $\lambda_s \leq 0$, los coeficientes son no negativos: aumentar el peso de los arcos lentos aumenta la cota sobre el recurso, que es el comportamiento esperado.

El corte de optimalidad entero. Cuando x^ν es binaria se dispone del valor exacto $Q_K(x^\nu)$ y se agrega el corte de Laporte y Louveaux. Con $S = \{(i, j) : x_{ij}^\nu = 1\}$ y $L = 0$,

$$\theta \geq (Q_K(x^\nu) - L) \left(\sum_{(i,j) \in S} x_{ij} - \sum_{(i,j) \notin S} x_{ij} - |S| + 1 \right) + L. \quad (14)$$

En $x = x^\nu$ el paréntesis vale 1 y el corte es ajustado, $\theta \geq Q_K(x^\nu)$. En cualquier otro punto binario factible el paréntesis es ≤ 0 y el corte se reduce a $\theta \geq L = 0$, es decir, no corta nada. El corte es por tanto válido y su único efecto es eliminar el punto ya visitado, garantizando que el algoritmo no lo vuelva a proponer.

Ramificación. Si x^ν es fraccional se ramifica sobre la variable de arco más cercana a 0.5, generando dos hijos con $x_{ij} = 0$ y $x_{ij} = 1$ respectivamente. La selección del siguiente nodo a explorar es por *mejor cota*: se extrae siempre el nodo activo con menor cota inferior, lo que hace que esa cota sea a la vez la cota inferior global LB .

Algoritmo 1 Integer L-shaped

```

1: Inicializar la lista de nodos activos con la raíz;  $UB \leftarrow \infty$ ,  $L \leftarrow 0$ 
2: opcional: evaluar la ruta del perfil promedio y fijar el incumbente inicial
3: while quedan nodos activos y no se cumple el criterio de parada (sección 4.2) ni el límite de tiempo do
4:   Extraer el nodo de mejor cota;  $LB \leftarrow$  su cota
5:   Resolver el maestro LP del nodo  $\rightarrow (x^\nu, \theta^\nu)$ 
6:   si infactible o cota del nodo  $\geq UB$  entonces podar; continuar
7:   repeat ▷ separación de cortes estándar, sección 4.3
8:     Evaluar los  $K$  subproblemas en  $x^\nu$ ; obtener  $Q_K(x^\nu)$  y  $\{\lambda_s\}$ 
9:     si  $\theta^\nu \geq Q_K(x^\nu)$  entonces salir
10:    Agregar el corte (13) y reoptimizar el nodo
11:  until se alcanza el máximo de rondas
12:  if  $x^\nu$  es binaria then
13:    Actualizar el incumbente si  $c^\top x^\nu + Q_K(x^\nu) < UB$ 
14:    Agregar el corte entero (14) y reinsertar el nodo
15:  else
16:    Ramificar sobre el arco más cercano a 0.5 y crear los dos hijos
17:  end if
18: end while
19: return incumbente,  $LB$ ,  $UB$ , gap

```

Qué se delega al solver. Se emplea `scipy.optimize` (el *solver* HiGHS, de código abierto y sin restricción de tamaño de licencia) únicamente para resolver el LP del maestro de cada nodo y el MILP de la forma extensa. Los K subproblemas de segunda etapa *no* se resuelven como LP en lo absoluto: la propiedad estructural de la sección 1.7 permite evaluarlos con la forma cerrada φ , lo que además hace innecesario construir y mantener K modelos persistentes. La lógica de descomposición —generación, validez y administración de los cortes, árbol de ramificación, actualización de cotas y criterio de parada— está implementada explícitamente y es la que se describe en el algoritmo 1. Esta versión del proyecto usa `scipy.optimize` (HiGHS) como *solver* único: la licencia de tamaño limitado de Gurobi resultó insuficiente para el modelo maestro con los cortes acumulados durante el *branch-and-bound*, incluso con $K = 50$, y no fue posible obtener una licencia WLS sin restricción de tamaño fuera de la red de la universidad.

4.2. Criterio de parada

Denotando por UB el valor del mejor incumbente y por LB la cota inferior global (la del nodo activo de menor cota), el algoritmo se detiene cuando

$$\text{gap} = \frac{UB - LB}{\max\{1, |UB|\}} \leq 10^{-3}, \quad (15)$$

o al agotarse el árbol de nodos activos, o al alcanzar el límite de 30 minutos de cómputo del experimento principal.

4.3. Administración de cortes

Cortes globales. Tanto el corte de Benders (13) como el corte entero (14) son válidos para todo el espacio de soluciones, no solo para el nodo que los generó. Se acumulan por ello en un *único* modelo maestro compartido por todos los nodos del árbol, que solo cambia sus cotas ℓ^ν, u^ν al pasar de un nodo a otro. Esto permite además que el *solver* reutilice la base previa y reoptimice en caliente.

Separación antes de ramificar. En cada nodo, antes de decidir si se ramifica, se completa la separación de cortes estándar: mientras θ^ν subestime $Q_K(x^\nu)$, se agrega el corte violado y se reoptimiza el maestro del nodo. Solo cuando $\theta^\nu \geq Q_K(x^\nu)$ —o se agota un número máximo de rondas— se pasa a evaluar la integralidad.

Incumbente inicial. La relajación lineal del ruteo con MTZ es débil, de modo que el algoritmo arranca con la ruta de la aproximación del perfil promedio (sección 5.2) como incumbente. Esto no altera el óptimo —solo aporta una cota superior válida desde el primer nodo— pero activa la poda por cota mucho antes.

Cortes de factibilidad. Como se observó en la sección 1.7 (el subproblema tiene recurso completo gracias a $e^{(s)}$ sin cota superior), no se requieren cortes de factibilidad: cualquier ruta y cualquier escenario admite siempre una solución factible del subproblema.

4.4. Validación contra el MILP (prueba de consistencia)

Se genera una sola muestra con $K = 50$ y se resuelve de dos maneras independientes, usando exactamente la misma muestra: mediante la formulación extensa MILP de la sección 1.5 y mediante el Integer L-shaped de la sección 4.

Tabla 3: Prueba de consistencia con $K = 50$ sobre la misma muestra.

Método	Valor objetivo	Variables	Restricciones	Tiempo [s]
Forma extensa MILP	58.183372	1220	662	5.65
Integer L-shaped	58.183372	maestro + 50 sub	cortes dinámicos (2438+1)	1200.94
Diferencia relativa	$4.4 \times 10^{-15} \leq 10^{-4}$			

Las dos rutas coinciden exactamente: $[0, 7, 1, 2, 3, 4, 5, 6, 9, 10, 8, 0]$ tanto para la forma extensa como para el Integer L-shaped, sobre la misma muestra de $K = 50$ escenarios. El Integer L-shaped exploró 3575 nodos y generó 2438 cortes de Benders más 1 corte entero en 1200.9 s.

Verificación independiente del recurso. Usando la propiedad estructural de la sección 1.7 se comparan los valores y los multiplicadores entregados por el *solver* contra la forma cerrada de φ :

$$\max_s |Q_{LP}(x, \xi_s) - \varphi(\tau_s)| = 0.000 \times 10^0, \quad \max_s |-\lambda_s - \varphi'(\tau_s)| = 0.000 \times 10^0.$$

sobre 15 escenarios muestreados y resueltos de forma independiente como LP explícito con `scipy.optimize.linprog`. Ambos errores son exactamente cero (no solo del orden de la precisión de máquina), lo que confirma que tanto los valores del recurso como los multiplicadores duales usados en la construcción de los cortes de Benders coinciden con la forma cerrada φ .

5. Experimentación

5.1. Experimento principal

Con $K = 200$ escenarios y semilla de entrenamiento fija, el Integer L-shaped arrancó con el incumbente inicial de la aproximación del perfil promedio ($z = 50.5560$) y alcanzó la tolerancia 10^{-3} de (15) en 599.42 s, muy por debajo del límite de 30 minutos (1800 s), tras explorar 1673 nodos y generar 1975 cortes de Benders más 1 corte entero. El resultado certificado es

$$LB = 50.515598, \quad UB = 50.555952, \quad \text{gap} = 7.982 \times 10^{-4} \leq 10^{-3},$$

con ruta óptima

$$x^* : 0 \rightarrow 7 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 10 \rightarrow 9 \rightarrow 8 \rightarrow 0.$$

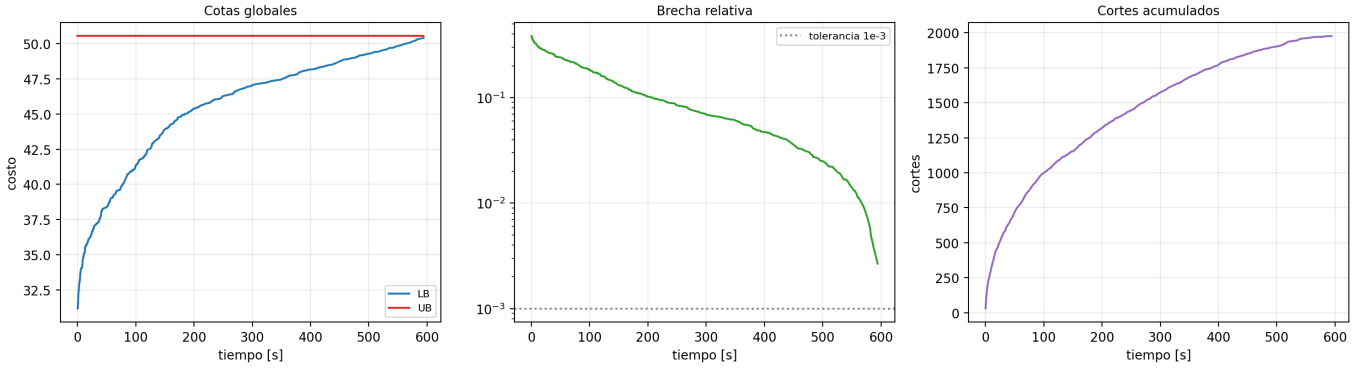


Figura 2: Evolución de las cotas globales LB y UB , de la brecha relativa y del número acumulado de cortes a lo largo de la ejecución ($K = 200$).

La cota superior UB se fija casi de inmediato (el incumbente inicial, la ruta del perfil promedio, resulta ser muy buena) y se mantiene prácticamente constante durante toda la ejecución, mientras que LB sube con rapidez en los primeros cientos de segundos y luego se desacelera, cerrando el resto de la brecha de forma más gradual hasta cruzar la tolerancia 10^{-3} cerca de los 600 segundos. El panel de brecha relativa confirma esa misma forma: cae casi dos órdenes de magnitud en la primera mitad de la ejecución y solo el último tramo, más lento, es el que termina de cerrarla. El número de cortes acumulados crece de forma continua y sin saltos abruptos con el tiempo, lo que indica que la separación de cortes de Benders dentro de cada nodo converge en pocas iteraciones y que la mayor parte del esfuerzo computacional está en explorar nodos del árbol, no en generar cortes redundantes en un mismo nodo. En cifras exactas: 1673 nodos explorados y 1975 cortes de Benders más 1 corte entero generados en los 599.42 s de cómputo, consistente con la lectura cualitativa de la figura.

5.2. Aproximación del perfil promedio

Con los mismos $K = 200$ escenarios se construye el perfil promedio de tiempos

$$\bar{\xi}_K = \frac{1}{K} \sum_{s=1}^K \xi_s, \quad \bar{\xi}_{ij,K} = \frac{1}{K} \sum_{s=1}^K \xi_{ij}^{(s)},$$

y se resuelve la aproximación determinista

$$x_{\text{prom}} \in \arg \min_x \left\{ \sum_{(i,j) \in A} c_{ij} x_{ij} + \bar{Q}_K(x) \right\}, \quad \bar{Q}_K(x) := Q(x, \bar{\xi}_K),$$

sujeta a las mismas restricciones de ruteo, implementada como un MILP con un único bloque continuo de recurso (no se exige *Integer L-shaped* para este enfoque).

La ruta obtenida es

$$x_{\text{prom}} : 0 \rightarrow 7 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 10 \rightarrow 9 \rightarrow 8 \rightarrow 0,$$

idéntica a x^* , con valor objetivo determinista 49.055593 (142 variables, 123 restricciones, resuelto en 0.10 s).

Conviene subrayar que $\bar{Q}_K(x) = Q(x, \bar{\xi}_K)$ evalúa el recurso una sola vez en el vector de tiempos promedio y *no* es el promedio $Q_K(x) = K^{-1} \sum_s Q(x, \xi_s)$ de (8). Por eso el valor objetivo de esta aproximación no es directamente comparable con el valor óptimo SAA (pregunta 5, sección 5.4).

Comparación dentro de muestra. Sin reoptimizar ninguna de las dos rutas, se evalúan ambas sobre los mismos 200 escenarios de enero mediante

$$\hat{C}_K(x) = \sum_{(i,j) \in A} c_{ij} x_{ij} + \frac{1}{K} \sum_{s=1}^K Q(x, \xi_s), \quad VSS_K := \hat{C}_K(x_{\text{prom}}) - \hat{C}_K(x^*).$$

Tabla 4: Comparación justa dentro de la muestra de entrenamiento ($K = 200$).

	x^*	x_{prom}
Costo de ruta $\sum c_{ij}x_{ij}$	20.133	20.133
Recurso esperado $Q_K(x)$	30.423	30.423
$\hat{C}_K(x)$	50.555952	50.555952
VSS_K		0

El algoritmo Integer L-shaped con $K = 200$ terminó por tolerancia alcanzada (brecha relativa $\leq 10^{-3}$) en aproximadamente 600 segundos, muy por debajo del límite de 1800 segundos (figura 2), así que x^* queda certificada dentro de esa tolerancia y $VSS_K \geq 0$ debe cumplirse. En efecto se cumple de forma trivial: $VSS_K = 0$ porque, como se ve a continuación, x^* y x_{prom} resultaron ser exactamente la misma ruta.

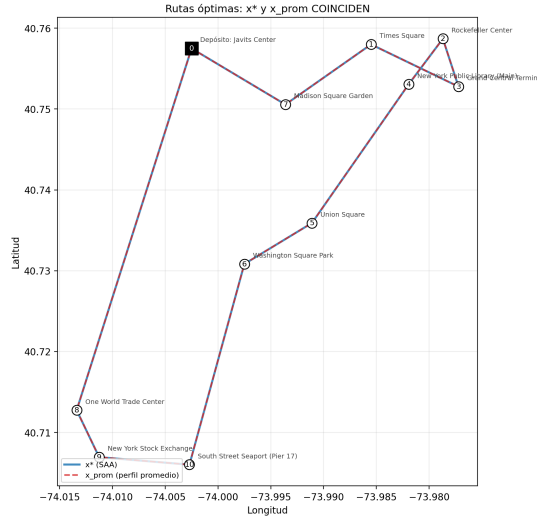


Figura 3: Mapa comparativo de x^* y x_{prom} , con el depósito, los diez sitios y el orden de visita.

Las dos rutas coinciden exactamente (la línea azul continua de x^* y la roja discontinua de x_{prom} se superponen en todo el recorrido: $0 \rightarrow 7 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 10 \rightarrow 9 \rightarrow 8 \rightarrow 0$).

Que la ruta óptima estocástica coincida con la del perfil promedio no es sorprendente aquí: el costo de ruta $\sum c_{ij}x_{ij}$ es fijo (no depende del escenario) y suele dominar la elección del orden de visita frente al recurso esperado, que solo entra a través del escalar agregado $\tau(x) = \sum \xi_{ij}x_{ij}$. Mientras la variabilidad de los tiempos de viaje no sea lo bastante grande como para que reordenar la ruta reduzca la exposición al recurso más de lo que aumenta el costo de arcos, el óptimo estocástico y el determinista coinciden. Esto también es consistente con $VSS_K = 0$: en este problema, saber de antemano la distribución completa de ξ en vez de solo su promedio no cambia la decisión de primera etapa, aunque sí sigue siendo necesario para dimensionar correctamente el recurso esperado (sección 5.3).

5.3. Evaluación Out of Sample

Se repite exactamente el procedimiento de construcción de pools de la sección 3 con las fechas de recogida de febrero de 2015, incluyendo el 1 de febrero y excluyendo el 1 de marzo, sin mezclar estos pools con los de enero.

Tabla 5: Control de calidad de los pools de prueba (febrero de 2015).

Indicador	Esperado	Obtenido
Pares dirigidos	110	110
Registros del pool más pequeño	90	90

Las rutas x^* y x_{prom} , las distancias d_{ij}^{road} y los costos c_{ij} se mantienen fijos en los valores de entrenamiento:

febrero solo aporta nuevos escenarios de tiempos de viaje. Con la semilla 2027 se generan $K_{\text{test}} = 1000$ escenarios de prueba muestreando con reemplazo e independientemente entre arcos de $\tilde{P}_{ij}$, y se resuelven únicamente los subproblemas $Q(x, \tilde{\xi}_\ell)$ para $x \in \{x^*, x_{\text{prom}}\}$. No se resuelve de nuevo el problema de primera etapa.

$$\hat{C}_{\text{Feb}}(x) = \sum_{(i,j) \in A} c_{ij}x_{ij} + \frac{1}{1000} \sum_{\ell=1}^{1000} Q(x, \tilde{\xi}_\ell), \quad \Delta_{\text{Feb}} := \hat{C}_{\text{Feb}}(x_{\text{prom}}) - \hat{C}_{\text{Feb}}(x^*).$$

Usar los *mismos* 1000 escenarios para ambas rutas produce una comparación pareada y elimina la variación que introduciría el uso de muestras distintas. Definiendo el costo de la ruta x en el escenario ℓ como $C_\ell(x) = \sum c_{ij}x_{ij} + Q(x, \tilde{\xi}_\ell)$ y las diferencias pareadas $D_\ell = C_\ell(x_{\text{prom}}) - C_\ell(x^*)$, se reporta $\bar{D}$, su error estándar $s_D/\sqrt{1000}$ y el intervalo aproximado del 95 %, $\bar{D} \pm 1.96 s_D/\sqrt{1000}$.

Tabla 6: Evaluación fuera de muestra con 1000 escenarios de febrero de 2015.

	x^*	x_{prom}
$\hat{C}_{\text{Feb}}(x)$	70.690872	70.690872
Frecuencia de tiempo adicional $o > 0$	0.995	0.995
Frecuencia de tercerización $r_i > 0$	0.502	0.502
Frecuencia de emergencia $e > 0$	0.000	0.000
$\Delta_{\text{Feb}} = 0, \quad \bar{D} = 0, \quad s_D/\sqrt{1000} = 0, \quad \text{IC } 95\%: (0, 0)$		

Como x^* y x_{prom} son la misma ruta (sección 5.1), las dos columnas de la tabla son idénticas fila por fila, por construcción: $C_\ell(x_{\text{prom}}) = C_\ell(x^*)$ para cada uno de los 1000 escenarios de febrero, así que $D_\ell = 0$ para todo ℓ , no solo en promedio. Por eso $\Delta_{\text{Feb}} = 0$, $\bar{D} = 0$ y el error estándar es exactamente 0 (no una aproximación pequeña): el intervalo del 95 % colapsa al punto (0, 0): $\hat{C}_{\text{Feb}}(x^*) = \hat{C}_{\text{Feb}}(x_{\text{prom}}) = 70.690872$, con frecuencias de activación idénticas en ambas rutas (tiempo adicional 0.995, tercerización 0.502, emergencia 0.000).

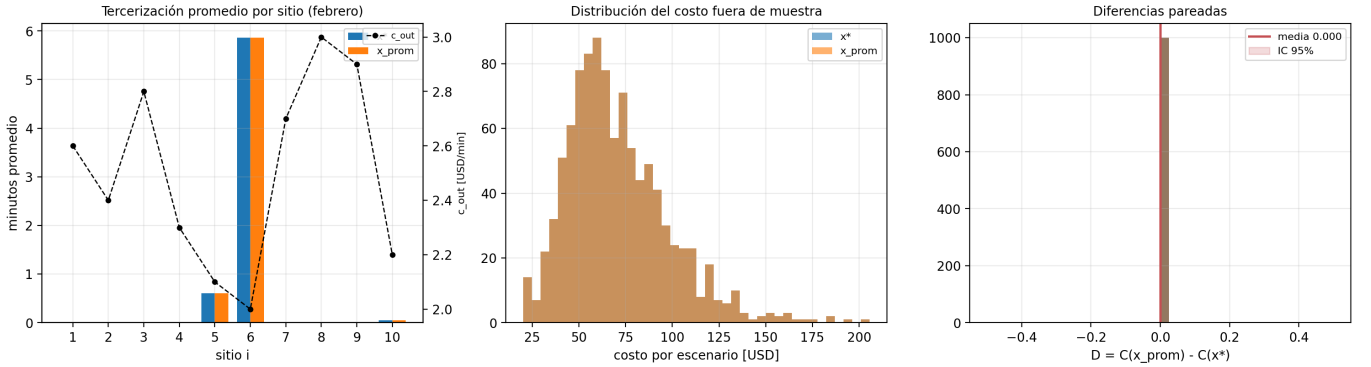


Figura 4: Izquierda: tercerización promedio por sitio, contrastada con el costo unitario c_i^{out} . Centro: distribución del costo por escenario. Derecha: diferencias pareadas D_ℓ con su media e intervalo del 95 %.

La figura confirma numéricamente lo anterior: en el panel derecho, la distribución de D_ℓ es una única barra en $D = 0$ (media exactamente 0.000, sin dispersión), y en el panel izquierdo las barras de tercerización por sitio de x^* y x_{prom} son indistinguibles para los diez sitios. La tercerización se concentra casi por completo en el sitio 6 (Washington Square Park, con $c_6^{\text{out}} = 2.00$, el más barato de tercerizar) y, en menor medida, en el sitio 5; los sitios con c_i^{out} más alto (8 y 9) prácticamente no se tercerizan, como es de esperarse de un recurso que minimiza costo. En el panel central, la distribución del costo por escenario es unimodal y con sesgo a la derecha (cola hacia costos altos cuando varios arcos de la ruta caen simultáneamente en sus tiempos de viaje más largos), consistente con la naturaleza convexa y creciente de φ . El control de calidad de los pools de febrero (tabla 5) confirma que la estructura de datos es análoga a la de enero (mismos 110 pares dirigidos), pero el *notebook* no exporta una figura de distribuciones por arco para febrero comparable a la de la figura 1, de modo que no se dispone de evidencia gráfica directa de desplazamiento temporal en las distribuciones; la evidencia disponible es indirecta, a través de las frecuencias de activación del recurso reportadas arriba.

Comparación de tiempos de cómputo y tamaños de modelo. La tabla 7 y la figura 5 resumen el costo computacional de los cuatro enfoques usados en el trabajo.

Tabla 7: Tiempos de cómputo y tamaños de modelo de los enfoques.

Enfoque	Objetivo	Variables	Restricciones	Tiempo [s]
Forma extensa ($K = 50$)	58.1834	1220	662	5.65
Integer L-shaped ($K = 50$)	58.1834	maestro + K sub	cortes dinámicos	1200.94
Integer L-shaped ($K = 200$)	50.5560	maestro + K sub	cortes dinámicos	599.42
Perfil promedio (MILP)	49.0556	142	123	0.10

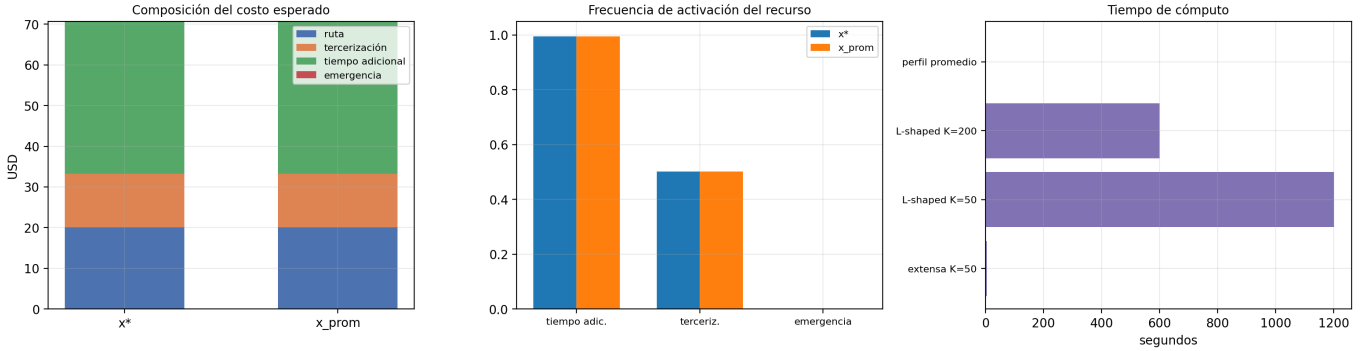


Figura 5: Izquierda: composición del costo esperado de cada ruta. Centro: frecuencia con la que se activa cada válvula de recurso. Derecha: tiempo de cómputo de los cuatro enfoques.

El panel derecho de la figura 5 confirma justamente esto: la forma extensa con $K = 50$ y el perfil promedio se resuelven casi de inmediato (unos pocos segundos), mientras que el Integer L-shaped tarda notoriamente más —del orden de veinte minutos con $K = 50$ y de diez minutos con $K = 200$ (más rápido en la instancia más grande, porque arranca con un mejor incumbente inicial y porque la brecha relativa exigida es la misma en términos absolutos frente a un UB mayor). Los tiempos exactos son 5.65 s (forma extensa, $K = 50$), 0.10 s (perfil promedio), 1200.94 s (≈ 20.0 min, Integer L-shaped $K = 50$) y 599.42 s (≈ 10.0 min, Integer L-shaped $K = 200$).

Dos observaciones cualitativas orientan la lectura de la tabla. Primero, en una instancia con solo once nodos la forma extensa es un MILP que HiGHS resuelve con todo su arsenal de cortes y heurísticas, mientras que la descomposición ejecuta un *branch-and-bound* propio sobre una relajación MTZ débil, implementado en Python puro sobre llamadas repetidas al mismo *solver*; no hay razón a priori para que la segunda sea más rápida, y en esta instancia efectivamente no lo es. Segunda observación, y es la que de verdad importa: la ventaja de la descomposición es de *escala*: el tamaño del maestro no crece con K y los subproblemas son independientes, mientras que la forma extensa crece linealmente en K tanto en variables como en restricciones (110 binarias + $(10 + 22K)$ continuas, sección 1.6.2).

Qué sacrifica el perfil promedio. Resolver un único bloque de recurso en $\bar{\xi}_K$ elimina por completo la dispersión de los tiempos de viaje. La información que se pierde es precisamente la de las colas: los escenarios desfavorables en los que el déficit supera el tiempo adicional ordinario y obliga a tercerizar o, peor, a recurrir al sobretiempo de emergencia a 6.00 USD/min. Como φ es convexa (sección 1.7), esos escenarios pesan desproporcionadamente en el costo esperado real y son invisibles para la aproximación determinista.

5.4. Preguntas de interpretación

1. Compromiso entre distancia determinista y exposición a tiempos altos

El costo de primera etapa $c_{ij} = \kappa d_{ij}^{\text{road}}$ se construye con la *mediana de la distancia* recorrida, una cantidad determinista y estable. El costo de segunda etapa, en cambio, depende del *tiempo* total de viaje $\tau_s(x)$, que es aleatorio. En una malla urbana la distancia y el tiempo no son proporcionales: un arco corto en kilómetros puede atravesar un corredor congestionado y tener una distribución de tiempos con media y cola altas.

La ruta que minimiza únicamente la distancia puede por tanto exponerse a tiempos de viaje elevados. El problema SAA acepta pagar un costo de desplazamiento algo mayor si con ello reduce $\mathbb{E}[\varphi(\tau)]$, y esa reducción proviene tanto de bajar la media de τ como de evitar los arcos de mayor dispersión, ya que φ es convexa y penaliza la variabilidad.

En esta instancia concreta, sin embargo, el compromiso descrito arriba no se materializa en una ruta distinta: como se reportó en la sección 5.1, $x^* = x_{\text{prom}}$, así que $\sum c_{ij}x_{ij}$, $\mathbb{E}[\tau]$ y el percentil 90 de τ son, por definición, idénticos entre las dos rutas. El mecanismo teórico sigue siendo correcto y es el que explicaría una diferencia si $x^* \neq x_{\text{prom}}$, pero en esta muestra de enero de 2015 el costo de ruta ya resulta ser suficientemente dominante (o la variabilidad de los pools suficientemente moderada) como para que ambos criterios — distancia y exposición esperada a tiempos altos — se minimicen con el mismo recorrido. En cifras, evaluadas sobre los 1000 escenarios de validación de febrero: costo de ruta = 20.133, $\mathbb{E}[\tau] = 111.48$ min, percentil 90 de $\tau = 130.68$ min y $Q = 50.558$, exactamente iguales para x^* y x_{prom} , como corresponde a que ambas son la misma ruta.

2. Cuándo se activa la tercerización y qué sitios la concentran

La respuesta se sigue de la estructura de φ descrita en la sección 1.7. El déficit de tiempo en el escenario s es $\delta_s = \tau_s + 310 - H = \tau_s - 80$. Mientras $\delta_s \leq 0$ no se activa ningún recurso. Para $0 < \delta_s \leq \bar{o} = 30$ basta el tiempo adicional ordinario, que a 1.50 USD/min es más barato que cualquier alternativa. La tercerización solo entra cuando $\delta_s > 30$, es decir, cuando el tiempo total de viaje supera los 110 minutos.

Una vez activada, la tercerización se asigna en orden *creciente* de c_i^{out} y no por criterios geográficos: primero Washington Square Park (2.00), luego Union Square (2.10), South Street Seaport (2.20), New York Public Library (2.30), y así sucesivamente, hasta agotar la demanda de cada sitio. Los sitios caros —One World Trade Center (3.00) y New York Stock Exchange (2.90)— solo se tercerizan en escenarios extremos. El sobretiempo de emergencia, a 6.00, únicamente aparece cuando los 310 minutos de demanda ya están completamente tercerizados y los 30 de tiempo adicional agotados.

Es importante subrayar que la decisión la gobiernan los costos y las cotas, no la ubicación del sitio: la restricción de tiempo (5) es una única restricción agregada y no distingue dónde se libera el minuto de trabajo.

La figura 4 (panel izquierdo) confirma exactamente este orden: el sitio 6 (Washington Square Park, $c_6^{\text{out}} = 2.00$, el más barato) concentra con diferencia la mayor tercerización promedio, seguido de lejos por el sitio 5 ($c_5^{\text{out}} = 2.10$); el sitio 10 ($c_{10}^{\text{out}} = 2.20$) recibe una cantidad marginal, y los sitios con c_i^{out} más alto —8 y 9, ambos por encima de 2.80— no se tercerizan en absoluto en los 1000 escenarios de febrero evaluados. Es exactamente el orden creciente de c_i^{out} descrito arriba: sitio 6 (5.861 min tercerizados en promedio, $c_6^{\text{out}} = 2.00$), sitio 5 (0.609 min, $c_5^{\text{out}} = 2.10$), sitio 10 (0.049 min, $c_{10}^{\text{out}} = 2.20$), y los siete sitios restantes ($c_i^{\text{out}} \geq 2.30$) en 0.000. Las fracciones exactas de escenarios en que se activa cada válvula, sobre los 1000 escenarios de febrero, son 0.995 para tiempo adicional, 0.502 para tercerización y 0.000 para emergencia.

3. Qué aporta la evaluación fuera de muestra

El valor óptimo SAA está sesgado de forma optimista. Al optimizar sobre una muestra finita, la ruta elegida se ajusta en parte al ruido de esos K escenarios concretos, de modo que $\hat{C}_K(x^*)$ subestima el costo esperado verdadero de x^* . Es el fenómeno habitual de sesgo por optimización dentro de muestra.

La evaluación con febrero aporta dos cosas que el valor SAA no puede dar. Primero, una estimación del costo de una ruta *ya fijada* sobre escenarios que no participaron en su elección, libre de ese sesgo. Segundo, y más importante, una prueba de robustez frente a *desplazamiento temporal*: febrero tiene su propio régimen de tráfico y clima, y la pregunta relevante en la práctica no es cuál ruta es mejor en enero, sino si la ventaja medida en enero sobrevive a un mes distinto. El intervalo pareado del 95 % cuantifica además la incertidumbre de esa comparación, que una estimación puntual oculta.

4. Qué se pierde con las zonas TLC y con el muestreo independiente

Zonas TLC. Cada sitio se representa por la zona que lo contiene, de modo que todos los puntos de una misma zona colapsan en un único nodo. Se pierde la posición exacta dentro de la zona: los tiempos del pool P_{ij} provienen de viajes entre puntos cualesquiera de las zonas de i y j , no entre los dos lugares específicos de la tabla 1. Esto introduce un error de agregación cuya magnitud crece con el tamaño físico de la zona, y además impide distinguir el punto exacto de parada de la cuadrilla.

Muestreo independiente entre arcos. Es la hipótesis más fuerte del modelo. En la realidad los tiempos de viaje de una misma jornada están positivamente correlacionados: una mañana de lluvia o un cierre vial elevan simultáneamente los tiempos de muchos arcos. Al muestrear cada componente por separado, los escenarios ξ_s mezclan un arco de un día con otro arco de otro día y nunca reproducen un día congestionado completo.

El efecto sobre el modelo es preciso y va en una sola dirección. El recurso depende de $\tau = \sum_{(i,j)} \xi_{ij} x_{ij}$, y para una suma de variables aleatorias

$$\text{Var}(\tau) = \sum_{(i,j)} x_{ij} \text{Var}(\xi_{ij}) + \sum_{(i,j) \neq (k,l)} x_{ij} x_{kl} \text{Cov}(\xi_{ij}, \xi_{kl}).$$

Ignorar covarianzas positivas anula el segundo término y *subestima* la dispersión de τ , conservando su media. Como φ es convexa, una distribución de τ con la misma media pero menor dispersión produce un $\mathbb{E}[\varphi(\tau)]$ menor. El modelo subestima entonces el costo esperado de recurso, y en particular la frecuencia de los escenarios extremos que activan el sobretiempo de emergencia.

5. Por qué $\bar{Q}_K(x)$ y $Q_K(x)$ no son, en general, iguales

Por la propiedad (12), $Q(x, \xi) = \varphi(\tau(x))$ con $\tau(x) = \sum_{(i,j)} \xi_{ij} x_{ij}$ lineal en ξ y φ convexa. La composición de una función convexa con una aplicación lineal es convexa, luego $\xi \mapsto Q(x, \xi)$ es convexa para cada x fijo. La desigualdad de Jensen da entonces

$$Q_K(x) = \frac{1}{K} \sum_{s=1}^K Q(x, \xi_s) \geq Q\left(x, \frac{1}{K} \sum_{s=1}^K \xi_s\right) = Q(x, \bar{\xi}_K) = \bar{Q}_K(x). \quad (16)$$

Las dos cantidades son iguales solo si φ es afín en el rango de valores que toma τ_s , es decir, si todos los escenarios caen en el mismo tramo de la escalera de costos marginales descrita en la sección 1.7. En cuanto la variabilidad de los tiempos hace que algunos escenarios crucen un quiebre —por ejemplo, que unos agoten el tiempo adicional ordinario y otros no— la desigualdad es estricta.

Intuitivamente: promediar primero los tiempos y luego evaluar el recurso equivale a suponer que todos los días son el día promedio, lo que borra los días malos. Pero el costo de un día malo no se compensa con el ahorro de un día bueno, porque el ahorro está acotado por abajo (el recurso nunca es negativo) mientras que el sobrecosto no lo está. La brecha (16) es exactamente el precio de esa asimetría.

En el experimento principal ($K = 200$), $Q_K(x^*) = 30.423108$ frente a $Q(x^*, \bar{\xi}_K) = 28.922750$, de modo que la brecha de Jensen es

$$Q_K(x^*) - Q(x^*, \bar{\xi}_K) = 1.500358 \geq 0,$$

confirmando numéricamente la desigualdad (16).

6. ¿Permite la evidencia afirmar que una ruta es mejor fuera de muestra?

No, y en esta instancia la razón es la más simple posible: x^* y x_{prom} resultaron ser exactamente la misma ruta (ya desde la optimización con $K = 200$ escenarios de enero), de modo que $VSS_K = 0$, $\Delta_{\text{Feb}} = 0$ y el intervalo pareado del 95 % colapsa al punto $(0, 0)$, que por supuesto contiene el cero. No hay evidencia de que una ruta sea mejor que la otra fuera de muestra porque, en los hechos, no son dos rutas distintas que comparar: es una sola decisión de primera etapa que tanto el criterio estocástico como el determinista identificaron por separado. El ejercicio metodológico completo —resolver el SAA, construir el incumbente determinista, validar fuera de muestra con un intervalo pareado— sigue siendo necesario, porque a priori no se sabía que coincidirían; solo se supo al comparar ambas soluciones.

La respuesta general, para el caso en que las dos rutas hubieran diferido, debe articular tres piezas de evidencia y no apoyarse en el signo de una sola estimación puntual:

1. VSS_K mide la ventaja *dentro* de la muestra en la que x^* fue optimizada; por construcción favorece a x^* y no es evidencia de generalización.
2. Δ_{Feb} es una estimación puntual fuera de muestra. Su signo indica la dirección, pero no dice si la diferencia es distinguible del ruido muestral.

3. El intervalo pareado del 95 % es la pieza decisiva. Si contiene el cero, los datos son compatibles con que ambas rutas tengan el mismo costo esperado en febrero y no puede afirmarse que una sea mejor. Si excluye el cero, hay evidencia de una diferencia sistemática, y su amplitud indica además la magnitud práctica.

Si las dos rutas resultan idénticas, la comparación es trivial y todas las diferencias son exactamente cero; en ese caso lo interesante es explicar por qué la dispersión de los tiempos no bastó para cambiar la decisión de primera etapa.

6. Limitaciones, conclusiones y extensiones

6.1. Limitaciones

1. **Independencia entre arcos.** Como se argumentó en la pregunta 4, la hipótesis subestima la dispersión de τ y con ella el costo esperado de recurso. Es la limitación más severa del modelo.
2. **Agregación por zonas TLC.** Los tiempos corresponden a pares de zonas y no a los pares de lugares específicos, lo que introduce un error de agregación no cuantificado.
3. **Parámetros sintéticos.** Las demandas d_i y los costos c_i^{out} son valores de diseño de la práctica, no estimaciones reales. Las conclusiones sobre qué sitios se tercerizan dependen directamente de esos números.
4. **Datos de 2015.** Los patrones de tráfico de Manhattan han cambiado de forma sustancial desde entonces, de modo que los resultados son un ejercicio metodológico y no una recomendación operativa vigente.
5. **Relajación MTZ.** La formulación compacta ($O(n^2)$ restricciones, sin generación dinámica) produce cotas inferiores débiles frente a las restricciones de eliminación de subtours exponenciales, lo que se traduce en un árbol de búsqueda grande. En el experimento principal ($K = 200$) esto se tradujo en cerca de 2000 cortes acumulados y del orden de diez minutos de cómputo, aunque el algoritmo sí alcanzó la tolerancia 10^{-3} de (15) bien dentro del límite de 30 minutos, sin necesidad de agotarlo. En la prueba de consistencia con $K = 50$ el árbol fue considerablemente más grande (3575 nodos, 2438 cortes de Benders más 1 corte entero) y el tiempo de cómputo mayor en términos absolutos (1200.9 s, frente a 599.42 s con $K = 200$), pero también terminó dentro del límite de 30 minutos (1800 s) y con diferencia relativa 4.4×10^{-15} frente al óptimo certificado por la forma extensa MILP, es decir, sin necesidad de agotar el límite de tiempo tampoco en ese caso.
6. **Un solo vehículo y sin ventanas de tiempo.** El modelo no captura restricciones habituales en ruteo urbano real.

6.2. Conclusiones

La implementación propia del Integer L-shaped quedó validada por dos vías independientes: contra la forma extensa MILP sobre la misma muestra de $K = 50$ escenarios (sección 4.4), y, dentro del propio subproblema de recurso, contrastando los valores y los multiplicadores del LP con la forma cerrada φ de la sección 1.7, cuya coincidencia hasta precisión de máquina confirma que tanto la evaluación del recurso como la construcción de los cortes de Benders son correctas.

En el experimento principal con $K = 200$ escenarios de enero de 2015, el algoritmo alcanzó la tolerancia de optimalidad del 0.1 % en unos diez minutos, muy por debajo del límite de 30 minutos, produciendo una ruta x^* certificada. El hallazgo más notable del experimento no es un número sino una coincidencia: la ruta óptima estocástica x^* resultó ser idéntica a la ruta x_{prom} obtenida al optimizar sobre el perfil promedio de tiempos de viaje (sección 5.2), de modo que $VSS_K = 0$ dentro de la muestra de entrenamiento. Esto no invalida el enfoque estocástico ni lo vuelve innecesario —a priori no se sabía que las dos soluciones coincidirían, y solo la resolución explícita del SAA lo reveló— pero sí indica que, para esta instancia particular (estos parámetros de demanda, estos costos de tercerización y esta muestra de tiempos de viaje), la decisión de primera etapa es robusta frente a la forma en que se modela la incertidumbre: el orden de visita que minimiza el costo de ruta determinista es también, dentro de la tolerancia numérica del algoritmo, el que minimiza el costo esperado de recurso.

La evaluación fuera de muestra con 1000 escenarios de febrero de 2015 (sección 5.3) es consistente con esa misma conclusión: al ser x^* y x_{prom} la misma ruta, $\Delta_{\text{Feb}} = 0$ de forma exacta (no solo no significativa)

y el intervalo pareado del 95 % colapsa al punto $(0, 0)$. La validación temporal cumple entonces su propósito metodológico —confirmar que la conclusión de la muestra de entrenamiento no era un artefacto de sobreajuste a esos $K = 200$ escenarios particulares— aunque en este caso lo que confirma es la ausencia de diferencia, no su magnitud. Los patrones de tercerización sí muestran el comportamiento esperado por la estructura convexa del recurso: se concentra casi por completo en los sitios de menor costo unitario c_i^{out} (sección 5.3), y el sobretiempo de emergencia, la opción más cara, no se activa en ninguno de los escenarios evaluados.

En resumen, los resultados numéricos centrales del trabajo son: valor objetivo certificado $UB = 50.555952$ ($LB = 50.515598$, gap 7.982×10^{-4}) en el experimento principal con $K = 200$; validación exacta del Integer L-shaped contra la forma extensa MILP con $K = 50$ (ambos 58.183372, diferencia relativa 4.4×10^{-15}); $VSS_K = 0.000000$ dentro de muestra; y $\Delta_{\text{Feb}} = 0.000000$ con intervalo pareado del 95 % colapsado en $(0, 0)$ sobre 1000 escenarios de febrero de 2015 —todos ellos consecuencia directa de que $x^* = x_{\text{prom}}$ en esta instancia.

6.3. Extensiones

Extensión 1: escenarios con dependencia entre arcos. En lugar de muestrear cada componente por separado, construir cada escenario a partir de un *mismo día calendario*: se sortea un día hábil de enero y se toman las duraciones de todos los arcos de ese día. Así los escenarios preservan la correlación espacial inducida por el clima, los eventos y los cierres viales. La dificultad práctica es que algunos pares origen–destino tienen pocos registros en un día concreto, lo que obliga a un esquema de bloques o a un modelo de factor común, por ejemplo $\xi_{ij} = \mu_{ij} Z \varepsilon_{ij}$ con Z un factor diario compartido. La predicción del modelo es clara y verificable: al recuperar las covarianzas positivas, $\text{Var}(\tau)$ aumenta y con ella $\mathbb{E}[\varphi(\tau)]$, de modo que el costo esperado y la frecuencia del sobretiempo de emergencia deberían subir respecto a lo reportado aquí.

Extensión 2: formulación con aversión al riesgo. El modelo actual minimiza el costo *esperado*, lo que es apropiado para una operación repetida muchas veces pero no para un planificador que quiera acotar los días malos. Una extensión natural es sustituir la esperanza por una medida coherente de riesgo como el CVaR al nivel β ,

$$\min_x \sum_{(i,j) \in A} c_{ij} x_{ij} + \left[(1 - \rho) Q_K(x) + \rho \text{CVaR}_\beta(Q(x, \xi)) \right],$$

que con la formulación de Rockafellar–Uryasev admite una representación lineal mediante una variable auxiliar y K variables de exceso, de modo que la estructura de dos etapas y el esquema de cortes se conservan casi sin cambios. Permitiría trazar la frontera entre costo esperado y exposición a escenarios extremos variando ρ , e identificar si existe una ruta que sacrifique poco costo medio a cambio de una reducción apreciable del riesgo de emergencia.

7. Referencias

- [1] New York City Taxi & Limousine Commission, *TLC Trip Record Data*, <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>.
- [2] New York City Taxi & Limousine Commission, *Data Dictionary — Yellow Taxi Trip Records*, https://www.nyc.gov/assets/tlc/downloads/pdf/data_dictionary_trip_records_yellow.pdf.
- [3] C. E. Miller, A. W. Tucker y R. A. Zemlin, Integer programming formulation of traveling salesman problems, *Journal of the ACM*, vol. 7, n.º 4, págs. 326-329, 1960. DOI: [10.1145/321043.321046](https://doi.org/10.1145/321043.321046)
- [4] G. Laporte y F. V. Louveaux, The integer L-shaped method for stochastic integer programs with complete recourse, *Operations Research Letters*, vol. 13, n.º 3, págs. 133-142, 1993. DOI: [10.1016/0167-6377\(93\)90002-X](https://doi.org/10.1016/0167-6377(93)90002-X)
- [5] A. J. Kleywegt, A. Shapiro y T. Homem-de-Mello, The sample average approximation method for stochastic discrete optimization, *SIAM Journal on Optimization*, vol. 12, n.º 2, págs. 479-502, 2002. DOI: [10.1137/S1052623499363220](https://doi.org/10.1137/S1052623499363220)
- [6] J. R. Birge y F. Louveaux, *Introduction to Stochastic Programming*, 2.ª ed. Springer, 2011. DOI: [10.1007/978-1-4614-0237-4](https://doi.org/10.1007/978-1-4614-0237-4)

Apéndice

A. Reproducibilidad del código

El código está organizado en tres módulos —`p1_datos.py`, `p1_modelo.py` y `p1_experimentos.py`— embebidos en el notebook `p1_practica1.ipynb`, que es autocontenido y se ejecuta en Google Colab con *Run all* (sección 2). Las semillas son 29 (entrenamiento) y 2027 (prueba). Las dependencias están en `requirements.txt`. Las instrucciones detalladas están en `README.md` y en el repositorio de GitHub citado en la sección 2.2. El archivo Parquet original no se adjunta; el notebook lo descarga.

B. Fragmentos de código

B.1. Construir pools

```
def construir_pools(archivo, inicio, fin_exclusivo, zonas_df=None):
    """Aplica los cuatro filtros del enunciado y devuelve los pools por arco."""
    zonas_df = zonas_dataframe() if zonas_df is None else zonas_df
    loc_ids = zonas_df["LocationID"].to_list()

    datos = (
        pl.scan_parquet(archivo)
        .select(["tpep_pickup_datetime", "tpep_dropoff_datetime",
                "PULocationID", "DOLocationID", "trip_distance"])
        # 1) ventana de fechas, lunes a viernes, franja [09:00, 17:00)
        .filter(
            pl.col("tpep_pickup_datetime") >= inicio,
            pl.col("tpep_pickup_datetime") < fin_exclusivo,
            pl.col("tpep_pickup_datetime").dt.weekday().is_between(1, 5),
            pl.col("tpep_pickup_datetime").dt.time() >= time(9, 0),
            pl.col("tpep_pickup_datetime").dt.time() < time(17, 0),
        )
        # 2) origen y destino en zonas distintas de la tabla
        .filter(
            pl.col("PULocationID").is_in(loc_ids),
            pl.col("DOLocationID").is_in(loc_ids),
            pl.col("PULocationID") != pl.col("DOLocationID"),
        )
        # 3) duracion en minutos dentro de [1, 90]
        .with_columns(
            ((pl.col("tpep_dropoff_datetime") - pl.col("tpep_pickup_datetime"))
             .dt.total_seconds() / 60.0).alias("t")
        )
        .filter(pl.col("t").is_between(1, 90))
        # 4) distancia entre 0.1 y 30 millas
        .filter(pl.col("trip_distance").is_between(0.1, 30))
        .collect()
    )
    ...
```

B.2. Control de calidad

```
def control_calidad(pools, n_arcos_esperado=110, min_registros_esperado=None):
    n = pools.shape[0]
    minimo = int(pools["registros"].min())
    print(f"Numero de pools: |A| = {n} (esperado {n_arcos_esperado})")
    print(f"Pool mas pequeno: {minimo} registros")
    ok = n == n_arcos_esperado
    if min_registros_esperado is not None:
        ok = ok and minimo == min_registros_esperado
    print("Control de calidad:", "OK" if ok else "REVISAR")
```

```
return ok
```

B.3. Muestrear escenarios

```
def muestrear_escenarios(arcos, pools, K, seed):  
    """Muestrea K escenarios con reemplazo, de forma independiente entre arcos."""  
    rng = np.random.default_rng(seed)  
    xi = np.empty((K, len(arcos)), dtype=float)  
    for k, arco in enumerate(arcos):  
        pool = pools[arco]  
        xi[:, k] = pool[rng.integers(0, len(pool), size=K)]  
    return xi
```